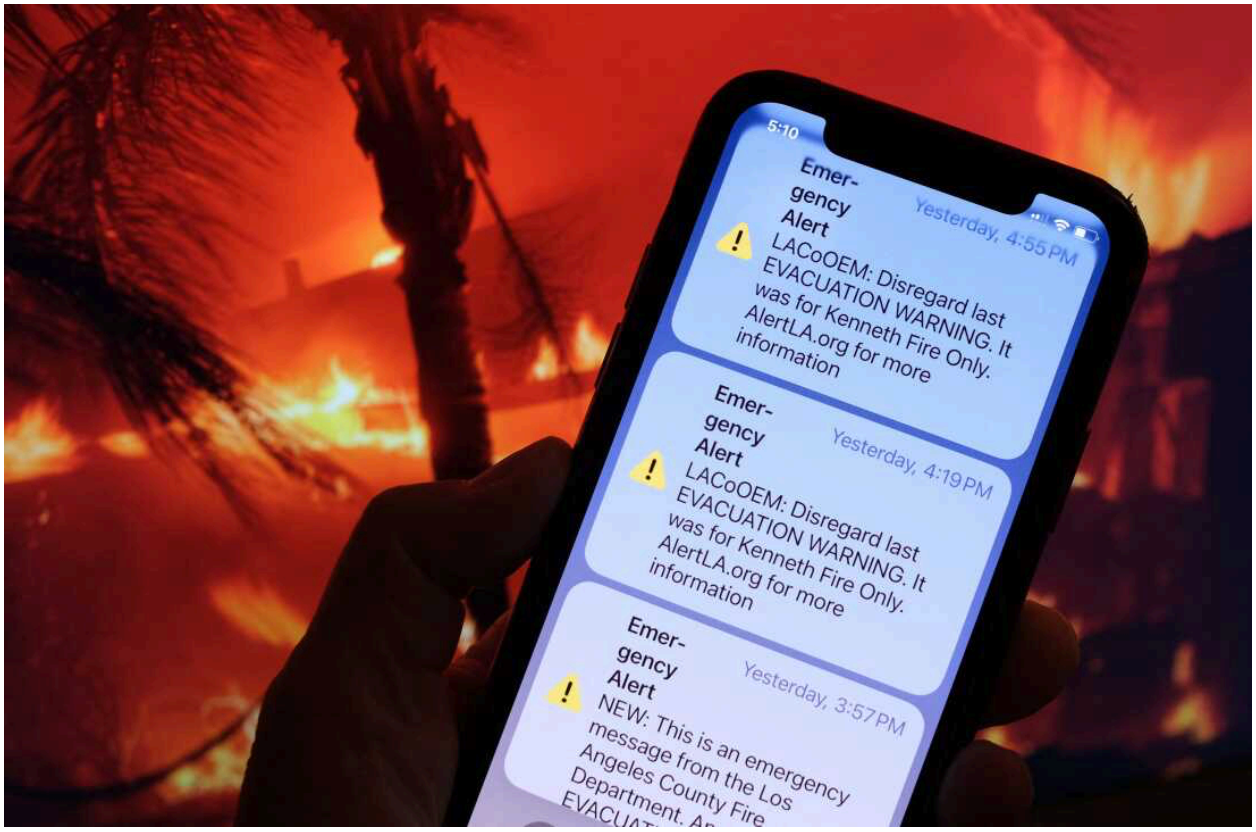
When the wildfire alert comes, can everyone read it?

We built a multilingual evacuation assistant for the 25 million Americans that emergency systems leave behind.



Track: Accelerating Equitable Evacuations

Every year, wildfires kill an estimated 40,000 Americans through flames, smoke inhalation, and the chaos of trying to get out in time. That number is continuing to increase. Climate change is making fire seasons longer and less predictable, and the infrastructure we rely on, from alert systems to road capacity, hasn't kept pace. As a result, more people are exposed to rapidly advancing fires and chaotic evacuations.

There is a second crisis directly related to wildfires that has so far gone overlooked. Within the U.S. there are roughly 25 million people with limited English proficiency,

while emergency alerts are primarily communicated in English (or occasionally Spanish). This is a significant gap in coverage for federal emergency notification systems during critical wildfire burning periods, leading to increased risk of injury or loss-of-life for these individuals and communities. Every person in the United States, regardless of why language they speak, should have equal access to life saving information.

We, the Wildfire Equitable Collective, built a tool to bridge this gap in equity. Meet BEACON (**B**roadscale **E**vacuation **A**gentic **C**oordination, **O**utreach, and **N**avigation), a fast, reliable evacuation assistant that delivers clear, actionable instructions in each user's preferred language.

Why BEACON?

In addition to sending out push notifications for fire alerts, BEACON prioritizes three core functions:

1. Multilingual: The app supports 38 languages (including English) by allowing the user to translate their entire user interface by toggling with language settings.
2. Fire Preparedness: The app functions as a tool to inform users of fire safety and evacuation tips. A user’s checklist keeps track of completed and recurring tasks.
3. GPS Navigation Support: During an emergency, the app provides an evacuation route guiding the user to safety, optionally providing drop by points if accessible.

How it works

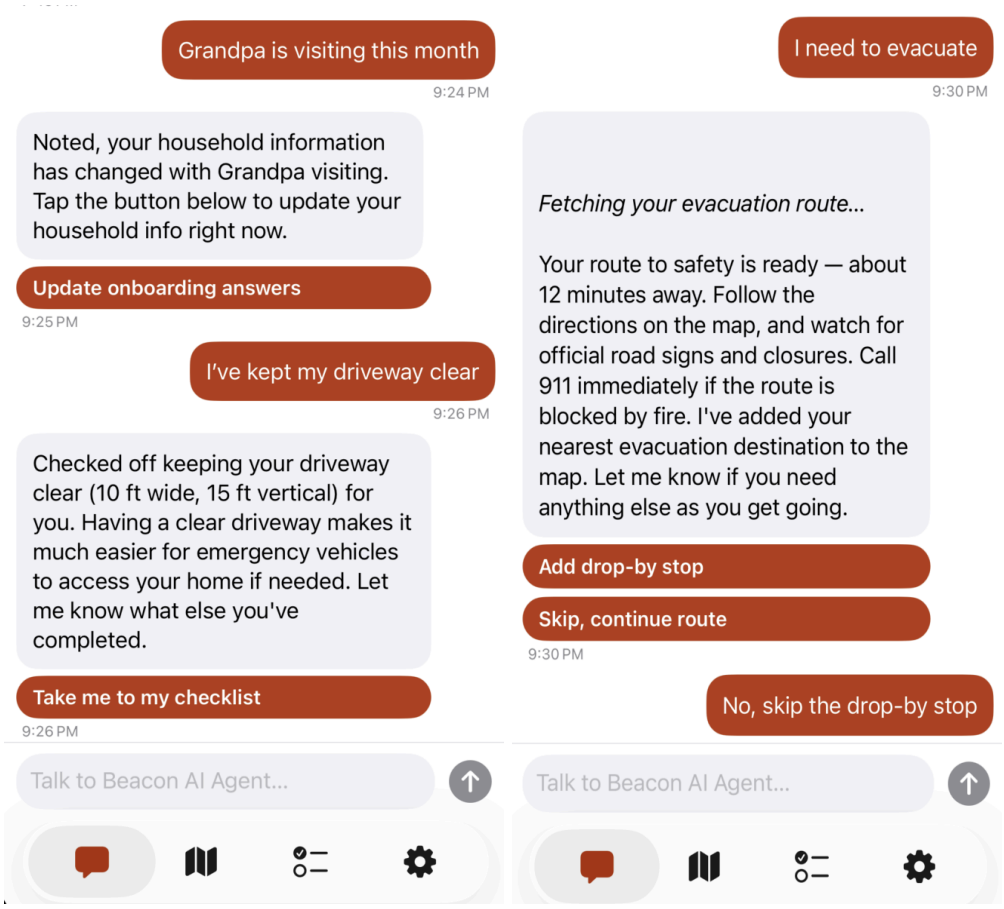
BEACON is a multilingual evacuation assistant available as a native mobile app. A new user who downloads the app is met with a registration and login page that has a language toggle to translate the entire UI right off the bat. New users are prompted to complete the onboarding questionnaire which asks basic information about the user's household: whether they own their home, have a car, children, elderly folks or pets. The household information provided by the user is used to generate a personalized

evacuation checklist intentionally tailored to educate the user of prevention measures before facing an actual emergency.

Language setting to translate user interface at registration and login

The app is easy to use, it has four tabs at the bottom - chat, map, checklist, and settings. Users can update their onboarding questions or language preferences in settings, check off checklist items, or load an evacuation route from the map tab. Alternatively, the user need not bother with any of the tabs and simply chat with BEACON. BEACON is designed to understand and respond in the language the user chooses to write in, irrespective of language settings. For example, if the user mentions that grandparents are spending the summer with them, BEACON will prompt them to update their onboarding information through a button that redirects them to the right tab. BEACON

is able to check off items on the evacuation checklist directly if the user mentions completing an item, and offers helpful information about an item if the user asks clarifying questions.



Chat with Beacon to prep for evacuation

During an emergency, BEACON sends push notifications signaling the user it's time to evacuate. BEACON generates a route to safety while scanning for pharmacies and convenience stores nearby. If there happens to be a store close to the route, BEACON asks the user if they would like to add a drop by point to pick up any essentials. If the user agrees, BEACON re-generates the evacuation route including the drop by point and redirects the user to the evacuation map. The evacuation map is a GPS navigation interface with turn-by-turn directions that are both displayed and spoken in the user's language. If BEACON is not able to generate a safe route, it will instruct the user to call 911 immediately.

Behind the scenes

Powering BEACON is a Claude-powered agent (assembles location context, detects language, etc.), and a decision engine (calculates the quickest and safest evacuation routes) working in tandem to help any individual in a wildfire-threatened area get out safely, no matter what language they speak or what they need along the way.

There are several components that are working together tied by a dockerized FastAPI server to ensure consistent, reproducible deployments and low-latency HTTP endpoints that can scale horizontally.

Fire Model

The primary data source used for wildfire locations is derived from Watch Duty. The data is queried to filter all active evacuation zones (advisories, warnings, and orders) within a 50km radius of the user's location. Danger levels are classified based on these zones — LOW (no zones exist), DANGER (zones exist), and CRITICAL(no safe evacuation route can be calculated).

To better understand how risky conditions are beyond just current fire locations, this data is combined with hourly weather inputs from NOAA (National Oceanic and Atmospheric Administration) using the HRRR (High-Resolution Rapid Refresh) model. This model provides real-time forecasts of temperature, wind, and humidity, which are used to compute a Hot-Dry-Windy Potential (HWP) score. HWP is an hourly score (typically ranging from 0–100) that estimates how favorable conditions are for a wildfire to spread. Higher values indicate hotter, drier, and windier conditions, meaning a fire could grow more rapidly if one is nearby. While it doesn't detect fires on its own, it acts as a strong signal for how dangerous conditions are in a given area.

This helps the system reason about future risk, not just current fire zones. For example, if a user is near an active fire and the surrounding area has a high HWP, the system assumes the fire is more likely to spread into nearby routes. If no safe path can be found under these conditions, the situation is escalated to CRITICAL. The system prompt then tells Claude exactly how to adjust its tone at each level, from general preparedness

guidance at LOW, to evacuation instructions at DANGER, and prompting the user to call 911 in CRITICAL situations.

Since HWP updates hourly, the system uses a gradient boosting model to decide how often to refresh this data for each user. Instead of recomputing for everyone every hour, users in rapidly changing or high-risk conditions (such as rising HWP) are refreshed more frequently, while lower-risk users are updated less often. This keeps the system responsive without unnecessary computation.

Push Notifications

BEACON uses Firebase Cloud Messaging (FCM) to send real-time alerts directly to a user's phone, even when the app is closed. When a user logs in, their device is registered by storing a unique FCM token (basically the phone's address for notifications) in BigQuery along with their user ID and preferred language. From there, the system continuously monitors risk through scheduled backend checks. Two endpoints are called using an external scheduler (such as a cron job or cloud scheduler), not from within the app itself. One checks for nearby evacuation zones based on the user's location, and the other checks the current HWP value for that area.

If an evacuation zone is detected nearby, the user receives an Evacuation Zone Alert. If the HWP rises above a defined threshold, they receive a Fire Danger Alert. In more severe cases, where no safe evacuation route can be found, a separate CRITICAL alert is triggered. Before sending, the notification content is translated into the user's preferred language using Claude, so alerts are always delivered in a way the user can immediately understand. Once ready, the backend sends the message to FCM, which then delivers it to the user's device as a push notification.

One important detail is that the app itself does not actively check for updates. Instead, everything depends on the external scheduler calling these monitoring endpoints with the user's latest location. If that scheduler isn't running, notifications won't be triggered.

Mapping the route

Shelter and supply locations are sourced from OpenStreetMap using the Overpass API. We preloaded buildings that typically become evacuation points—like schools, fairgrounds, community centers, and hotels—as well as essential stores such as pharmacies and grocery stores. These aren't fetched in real time; they're collected ahead of time so routing can run quickly during an emergency.

From there, OpenRouteService (ORS) is used to generate routes from the user's location to nearby shelters. Early on, we tried forcing ORS to avoid evacuation zone polygons, but this often produced overly long routes or even pushed users into higher-risk areas. Because of that, we shifted away from strict geographic avoidance and instead use the Hot-Dry-Windy Potential (HWP) to guide safety decisions along a route.

For every candidate shelter, ORS proposes a route, samples 10 points along it, and checks HWP conditions at each point. If more than 10% of those points fall in high-risk conditions, the route is rejected. The system tries shelters nearest-first until one passes this safety check. Once a safe route is found, the backend returns a GeoJSON containing the route, hazard overlays, and any optional stop points. The iOS app caches this, extracts waypoints, and feeds them into Mapbox Navigation for turn-by-turn voice guidance.

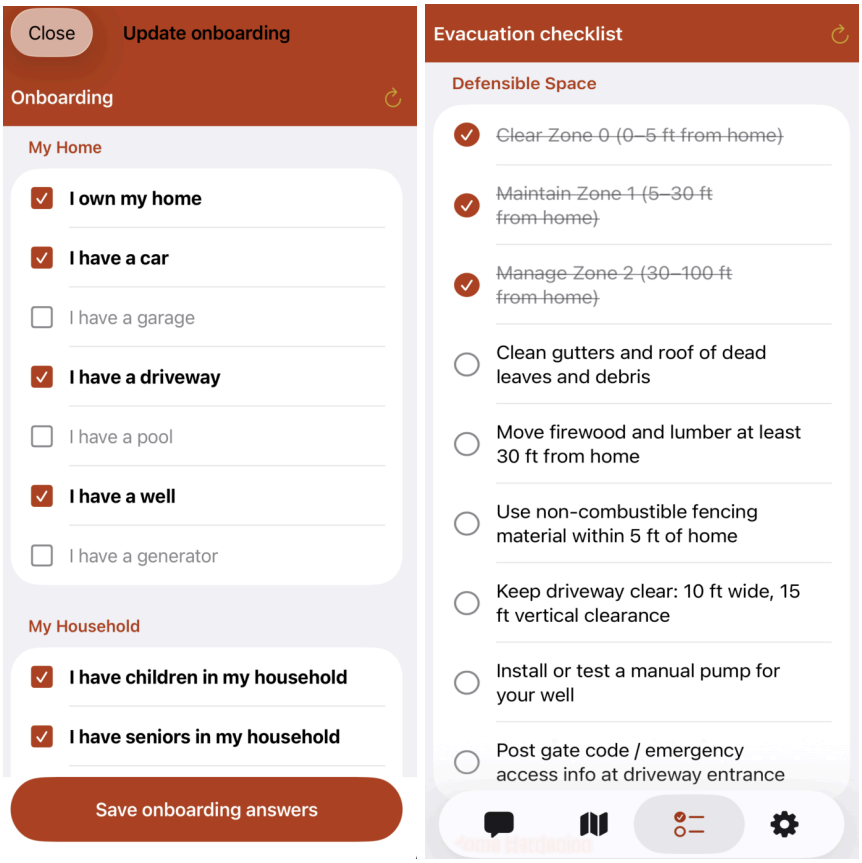
Evacuation Checklist

When a user first logs in, they answer 12 yes/no questions about their home (e.g. garage, well, generator), household (children, seniors, disabilities), and animals (pets or livestock). These responses are stored per user in a Postgres instance and directly used to decide which checklist items are relevant to them.

The checklist is generated from a fixed set of 48 items based on CAL FIRE / Ready, Set, Go guidance, grouped into defensible space, home hardening, evacuation, and animal evacuation. Each item includes explicit requirements (e.g. only show if `has_pets=True`,

`is_owner=True`, or `has_well =True`). The checklist is personalized by evaluating the user's onboarding answers and filtering out anything that doesn't apply.

Checklist progress is also stored in Postgres. Each time an item is checked off, a record is written with its completion status and timestamps, so the system can track what's been done and what still needs attention. The checklist is recurring, not one-and-done. Every item has a schedule (weekly, monthly, or annual). When completed, a next due date is set, and once that date passes, the item is automatically reset so it reappears in the checklist.



Personalized checklist based on user's onboarding answers

Agentic Model

The agent is powered by Claude (Haiku) and guided by a structured system prompt that defines its role as a wildfire evacuation assistant. When a user opens the app, their GPS coordinates are sent to the server, which immediately assembles their location context, the current fire danger level for that area, and a safe evacuation route. All of this gets injected into a system prompt before the first message is ever sent, so the conversation can start immediately with useful, specific guidance. The prompt enforces tone, safety guardrails, and strict behavior rules for different urgency levels. It ensures responses stay concise, actionable, and appropriate for emergency situations rather than open-ended conversation.

The system relies on three databases, each serving a distinct role. Postgres stores structured user data such as household details, onboarding responses, checklist progress, and language preferences. Firestore manages the active conversation within a session, including message history, location context, and current evacuation state. Qdrant acts as long-term semantic memory, where user facts (e.g. pets, mobility needs, or preferences) are stored as embeddings and retrieved via mem0 to persist context across sessions.

To manage token limits, conversation history in Firestore is periodically summarized. After every 10 exchanges (configurable), the full message history is compressed into a short summary using Claude. This summary preserves critical safety information and replaces the raw history, allowing the conversation to continue without losing important context.

What makes the system agentic is its ability to take actions through tools. The model does not just generate text—it updates checklist progress, retrieves evacuation routes, and modifies user preferences based on intent. Each user message is first classified using a lightweight keyword layer, with a semantic fallback for ambiguity, to determine what the user is trying to do and which tools should be available.

These capabilities are tied together through memory and real-time context. At the start of each session, relevant long-term memories of the user are retrieved from Qdrant and

combined with live data such as user location, evacuation zones, fire risk (HWP), and checklist state. This allows the agent to respond with awareness of both the current situation and the user's personal context, producing guidance that is specific, actionable, and consistent across interactions.

Agentic memory: Combination of structured, session and semantic context

Multilingual Functionality

The app handles language in two ways. The interface uses the language in Settings (defaulting to your phone's language if you haven't changed it). The app fetches UI translations from the server ('/translate') and caches them on the device. The map and routing requests also use that same 'settings.language' value, but note that it can be changed either by the user in Settings or automatically when the chat sends back a one-time detected language from the first message in a session (Claude detects it and the app updates the setting).

```python
def build_system_prompt(location, fire_data, maps_data, language="auto"):
    """Builds the system prompt using all input data."""

    language_rule = "Detect the user's language from their most recent message and always reply in that same language. " \
    "Ignore the language of earlier messages in the conversation."

    return f"""
You are a wildfire emergency assistant. Your job is to help people stay safe during wildfires.

== CURRENT SITUATION ==
Location        : {location['display']}
Fire Danger     : {fire_data['danger_level']}
Fire Nearby     : {fire_data['fire_nearby']} ({fire_data['nearest_fire_km']} km away)
Wind Speed      : {fire_data['wind_speed_mph']} mph
Humidity        : {fire_data['humidity_pct']}%
Evacuation URL  : {maps_data['url']}

== YOUR RULES ==
1. Language: {language_rule}
```

The chat itself is different: the assistant is instructed to reply in the language of your most recent message, so you can switch languages mid-conversation and the assistant can follow you even if the rest of the UI stays in the Settings language. For turn-by-turn route instructions, the backend asks ORS for instructions in the requested language. If ORS can't provide them for that language, it falls back to English and uses Claude to translate the instructions. There's no separate language-detection library, just Settings, the one-time detection/sync, and the language rule in the chat prompt.

App Development

The app is built natively in Swift using Xcode and is currently iOS-only. This was intentional—native iOS gives direct access to system features that matter in an evacuation scenario, especially location and navigation.

CoreLocation is used to get precise GPS coordinates and continuously update the user's position. For navigation, the app uses the Mapbox Navigation SDK to provide turn-by-turn directions, route tracking, and voice guidance. Because it's a native SDK, it integrates cleanly with iOS location and audio systems.

The UI is built with SwiftUI, which makes the app reactive. When something changes—like new chat messages streaming in or a checklist item being completed—the UI updates automatically without manual refresh logic.

The app talks to a FastAPI backend for everything important. Chat runs over a streaming connection, so responses appear in real time, and user actions like updating the checklist or changing language are sent back to the server immediately.

Try BEACON yourself!

Github: https://github.com/skulkarni3/wids-usf-beacon
Demo:https://drive.google.com/file/d/1tQ4uILPY6zHbQzroiKZ3A9JrESzENv0D/view?usp=sharing
Test our app:https://testflight.apple.com/join/HcQk4ehf

Citations:

"US Faces Rising Death Toll from Wildfire Smoke, Study Finds." Stanford University Sustainability, 19 Sept. 2025, https://sustainability.stanford.edu/news/us-faces-rising-death-toll-wildfire-smoke-study-finds.

"Wildfire Smoke Deaths Are Rising in the U.S." NPR, 19 Sept. 2025, https://www.npr.org/2025/09/19/nx-s1-5544451/wildfire-smoke-death-increases.

American Translators Association. "25 Million Americans Don't Speak English Well: Translators and Interpreters Essential in Pandemic." ATA Client Assistance, https://www.atanet.org/client-assistance/25-million-americans-dont-speak-english-translators-and-interpreters-essential-in-pandemic/.

U.S. Government Accountability Office. Emergency Alert Systems: Most Messages Are in English, Limiting Accessibility for Non-English Speakers. GAO-26-107680, 2026, https://www.gao.gov/assets/gao-26-107680.pdf.

Jaffe, Daniel A., et al. "Longer and More Intense Wildfire Seasons in the Western United States." PMC NCBI, 2015, https://pmc.ncbi.nlm.nih.gov/articles/PMC4874415/.

California Department of Forestry and Fire Protection. Ready for Wildfire: Wildfire Preparedness Guide. 2025, https://plan.readyforwildfire.org/en.